Lab M9.04 – Sharding and Horizontal Partitioning Strategy

Problem Statement

Given the following datasets, design a sharding strategy:

1. User accounts (global application)
   - Millions of users worldwide
   - Queries by user_id, email, username
   - High read and write volume

2. Transaction records (high volume)
   - Billions of transactions
   - Queries by transaction_id, user_id, date
   - Append-only, time-series pattern

3. IoT sensor data (time-series)
   - Millions of devices
   - Queries by device_id, timestamp, location
   - High write volume, append-only

4. Orders by region (geographic distribution)
   - Global e-commerce platform
   - Queries by order_id, customer_id, region, date
   - Mixed read/write patterns

Step 1: Sharding Concepts Review

What is horizontal vs vertical partitioning?

Horizontal partitioning, "Sharding", splits data row-by-row. It has high scalability as we can add as many servers as we want. It is used to balance the data and the writes.

Vertical partitioning splits data column-by-column. Limited scalability. Normally used when there are columns that are rarely queried or are heavy.

What is the difference between sharding and indexing?

Sharding (also called horizontal partitioning) is the practice of splitting a large database table into smaller, independent pieces called shards. Whilst by indexing does not split but points. When there is an indexed column, instead of scanning the whole table, the column is just scanned to find the values we want to filter, join or sort.

What are the key considerations for sharding?

There are three options for sharding: hash-based and range-based or composite. Each type could be selected depending on the data volume, write patterns, read patterns and query frequency.

A proper shard key is the one that has high cardinality, distributes data and writes evenly and matches with most common queries.

Step 2: Shard Key Selection

Dataset 1: User Accounts

```
users (
  user_id INT PRIMARY KEY,
  email VARCHAR(100),
  username VARCHAR(50),
  country VARCHAR(50),
  created_at TIMESTAMP
)
```

Proposed shard key: user_id. Hash-based.

High cardinality: high, every user has a unique_id

Even distribution: high with hashing

Query alignment: medium. If there are filters by users, the shard key match the filter but if there are filters by email or username, cross shard-queries will be needed

Write distribution: high

Dataset 2: Transaction Records

```
transactions (
  transaction_id BIGINT PRIMARY KEY,
  user_id INT,
  amount DECIMAL(10,2),
  transaction_date TIMESTAMP,
  merchant_id INT
)
```

Proposed shard key: user_id. Hash-based.

High cardinality: high, every transaction has a unique_id

Even distribution: high with hashing

Query alignment: medium. If there are filters by user, the shard key match the filter but if there are filters by transactions, cross shard-queries will be needed

Write distribution: high

Dataset 3: IoT Sensor Data

```
sensor_readings (
  reading_id BIGINT PRIMARY KEY,
  device_id INT,
  sensor_type VARCHAR(50),
  value DECIMAL(10,4),
  timestamp TIMESTAMP,
  location VARCHAR(100)
)
```

Proposed shard key: device_id range + timestamp range combination (composite)

High cardinality: high with combination of both

Even distribution: balanced

Query alignment: high and flexible. Queries filter by device_id and time

Write distribution: medium as it is not 100% even distribution

Dataset 4: Orders by Region

```
orders (
  order_id BIGINT PRIMARY KEY,
  customer_id INT,
  region VARCHAR(50),
  order_date DATE,
  amount DECIMAL(10,2),
  status VARCHAR(20)
)
```

Proposed shard key: region, range-based

High cardinality: medium

Even distribution: medium, depending on how many orders per region

Query alignment: high. Queries grouped by region

Write distribution: medium as it is not 100% even distribution

Step 3: Risk Analysis

Dataset	Hot Shards Risk	Rebalancing Risk	Query Routing Risk	Data Skew Risk	Mitigation Strategies
User Accounts	None	None	Potential cross-shard querie	None	indexing
Transactions	None	None	Potential cross-shard querie	Very low since there are billions of transactions	indexing
IoT Sensor Data	None	Low, in case of uneven distribution	None, flexible	Very low as it is balanced	
Orders by Region	None, unless most orders are concentrated in a few number of regions	Low, in case of uneven distribution	None, range queries efficient	Low. Depending on orders per region.	Resharding, splitting, load balancing

Step 4: Final Design Summary

Dataset	Shard Key	Strategy	Primary Benefit	Main Risk	Mitigation
User Accounts	user_id	hash-based	even distribution	potential cross-shard queries	indexing
Transactions	user_id	hash-based	even distribution	potential cross-shard queries	indexing
IoT Sensor Data	device_id + timestamp	composite	balanced and flexible	complexity	
Orders by Region	region	range-based	query alignment	uneven distribution	resharding, splitting, load balancing